

Multithreading in C/C++

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory
Department of Electrical and Computer Engineering
Cullen College of Engineering
University of Houston

UNIVERSITY of **HOUSTON** | ECE

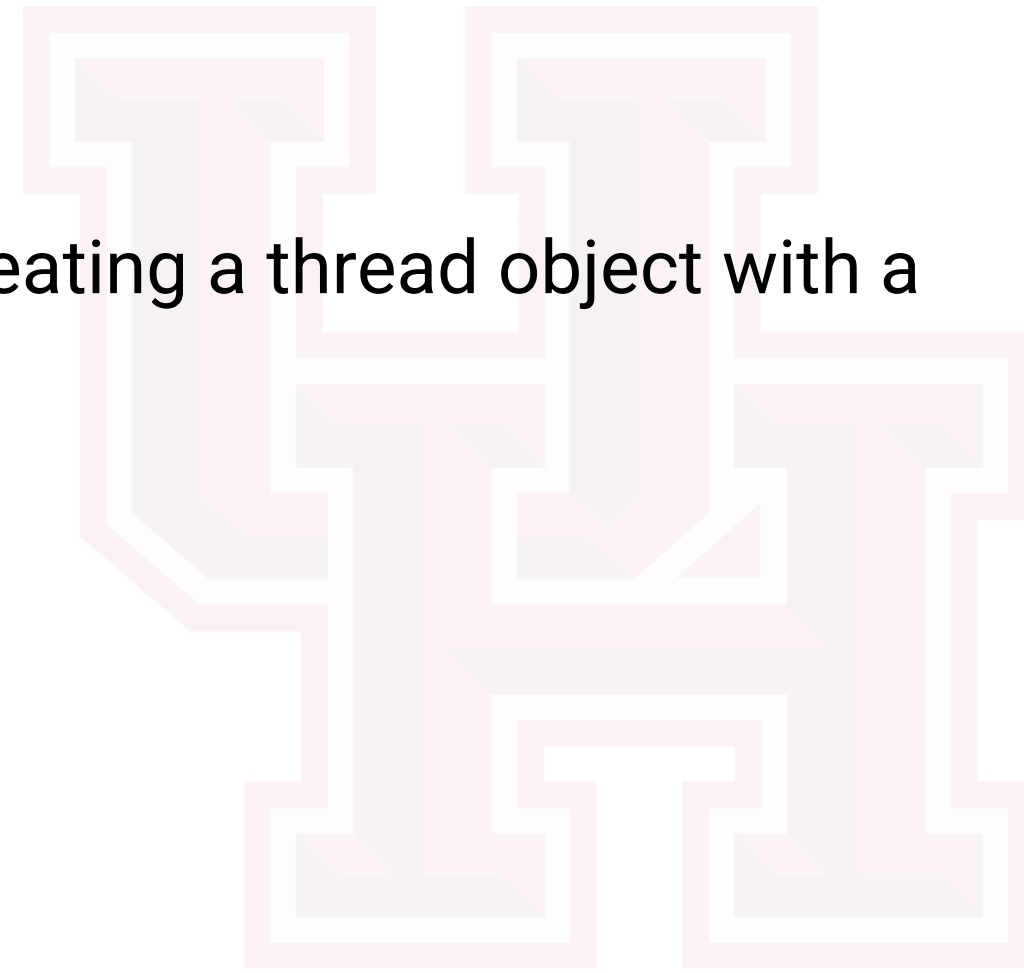
C++ Thread Library

- C++11 standard supports threads:

```
#include <thread>
```

- Independent threads can be launched by creating a thread object with a function pointer:

```
void func(float param){  
    // do independent work  
}  
  
// ...  
std::thread t(func, param);
```



Multithreading Example

- Consider a matrix-vector multiplication:

$$\mathbf{y} = \mathbf{M}\mathbf{x} = \begin{bmatrix} m_{1,1} & \cdots & m_{1,N} \\ \vdots & \ddots & \vdots \\ m_{N,1} & \cdots & m_{N,N} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix}$$

- This can be computed as a set of N dot (inner) vector products:

for $i = 1$ to N :

$$y_1 = \mathbf{m}_1 \cdot \mathbf{x} = [m_{1,1} \quad \cdots \quad m_{1,N}] \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix}$$

Write a Kernel Function

```
void dot(float* y, float* M, float* x, float row, float N){  
    float r = 0;           // initialize an internal register  
    for(int i = 0; i < N; i++){ // for each element in the vector  
        r += M[row * N + i] * x[i]; // calculate the inner product  
    }  
    *y[r] = r;             // store the result in y  
}
```

- Why use a new register r to accumulate the answer?
 - Memory accesses are slow, registers are fast

Implement Multithreading

- Define callback function:

```
void dot(float* y, float* M, float* x, float row, float N);
```

- Write single-threaded code:

```
float* x = new float[N];  
float* y = new float[N];  
float* M = new float[N * N];  
std::thread* t = new std::thread[N];  
// fill memory with values  
for(int ri = 0; ri < N; ri++){  
    dot(y, M, x, ri, N);  $\longrightarrow$  t[n] = std::thread(dot, y, M, x, ri, N);  
}
```

Synchronization

```
for(int ri = 0; ri < N; ri++){  
    t[ri] = std::thread(dot, y, M, x, ri, N);  
}
```

- Loop will return immediately
- HOWEVER, `y` will be invalid until all threads complete
- Synchronize with `join()`:

```
for(int ri = 0; ri < N; ri++){  
    t[ri].join();           // wait for thread t[ri] to complete  
}  
// if you are here, all threads are finished
```